

Week 1: Network Fundamentals

1. Services Offered by the Internet (10) - Applications

The internet provides a vast range of services that have become integral to modern life. Here are ten fundamental services and their common applications:

1. World Wide Web (WWW)

This service allows users to access and interact with a massive collection of interlinked documents and web resources.

- **Examples:** idomavoice.com, wikipedia.org
 - **Applications:** Browsing websites for information, reading news, accessing educational materials, and using web-based software.
-

2. Electronic Mail (E-mail)

E-mail is one of the most widely used services for sending and receiving digital messages. It has largely redefined communication by making it faster and more efficient than traditional mail.

- **Examples:** encyclopedia.com, edn.com
 - **Applications:** Personal communication, business correspondence, marketing, and sharing documents and files.
-

3. File Transfer

This service allows for the downloading and uploading of files between computers.

- **Examples:** edn.com

- **Applications:** Downloading software, sharing large documents, and accessing media files.
-

4. Communication and Collaboration

The internet enables real-time interaction between users across the globe.

- **Examples:** naukri.com
 - **Applications:** Instant messaging (e.g., WhatsApp), video conferencing (e.g., Zoom, Google Meet), and Voice over Internet Protocol (VoIP) for phone calls.
-

5. E-commerce

This refers to the buying and selling of goods and services online. E-commerce platforms have revolutionized the retail industry.

- **Examples:** quora.com, naukri.com
 - **Applications:** Online shopping on sites like Amazon and eBay, online banking, and digital payment services like PayPal.
-

6. Social Media

These platforms have become virtual meeting places, connecting people and facilitating the sharing of information and user-generated content.

- **Examples:** idomavoice.com
 - **Applications:** Networking on platforms like Facebook and Twitter, sharing photos and videos on Instagram and YouTube.
-

7. Entertainment and Streaming Media

The internet provides on-demand access to a vast library of entertainment content.

- **Examples:** wikipedia.org, naukri.com
 - **Applications:** Streaming movies and TV shows on services like Netflix, listening to music on Spotify, and playing online games.
-

8. Cloud Computing and Storage

Cloud services offer remote data storage and access to applications from anywhere with an internet connection.

- **Examples:** idomavoice.com, computerhope.com
 - **Applications:** Storing files on Google Drive or Dropbox, using online productivity suites like Google Docs, and hosting websites on platforms like AWS.
-

9. Education and E-learning

The internet has made education more accessible through online courses and resources.

- **Examples:** edn.com, naukri.com
 - **Applications:** Taking online courses on platforms like Coursera and Udemy, accessing virtual classrooms, and conducting research.
-

10. Information Retrieval (Search Engines)

Search engines have transformed how we find information, providing quick access to answers and data.

- **Examples:** idomavoice.com

2. Emerging Trends and Services

The internet is in a constant state of evolution, with new technologies and services continually reshaping the digital landscape. Here are some of the most significant emerging trends:

1. Artificial Intelligence (AI) and Machine Learning

AI is becoming deeply integrated into web services, powering everything from smarter search engines and personalized content recommendations to AI-driven chatbots for customer service. Generative AI, in particular, is transforming content creation across text, images, and audio.

- **Examples:** glofiber.com
-

2. Internet of Things (IoT)

The IoT ecosystem is rapidly expanding, connecting everyday objects like smart home devices, wearables, and industrial sensors to the internet. This creates a seamless network where devices communicate and share data, leading to smarter homes, more efficient cities, and advancements in areas like telemedicine.

- **Examples:** e8internet.com
-

3. 5G and Future Wireless Technologies

The rollout of 5G networks, and the development of future 6G technology, promise significantly faster speeds, lower latency, and greater capacity. This will enhance existing applications like high-definition streaming and online gaming, and enable new innovations such as autonomous vehicles and remote surgery.

- **Examples:** e8internet.com
-

4. Immersive Experiences (AR/VR/XR)

Augmented Reality (AR), Virtual Reality (VR), and Extended Reality (XR) are merging the physical and digital worlds. These technologies are moving beyond gaming into applications like virtual meetings, online education, and immersive shopping experiences where users can "try on" products virtually.

- **Examples:** forbes.com
-

5. Decentralization and Web3

There is a growing movement towards a decentralized internet, often called Web3, built on technologies like blockchain. This aims to shift control from large corporations to individual users, enhancing privacy, security, and data ownership.

- **Examples:** forbes.com
-

6. Edge Computing

Instead of processing data in centralized clouds, edge computing brings computation and data storage closer to the sources of data. This reduces latency and is crucial for real-time applications like IoT and autonomous systems.

- **Examples:** extremenetworks.com
-

3. Client-Side Components

Client-side refers to everything that is processed on the user's device, primarily within the web browser. These technologies are responsible for the **user interface (UI)** and creating an **interactive experience**.

Web User-Based Technologies

- **HTML (HyperText Markup Language):** The fundamental building block of the web, used to structure the content of a webpage.
- **CSS (Cascading Style Sheets):** Used to control the presentation, formatting, and layout of the content structured by HTML.
- **JavaScript:** A programming language that brings interactivity to a website. It handles dynamic content updates, animations, and user interactions without needing to reload the entire page.

- **JavaScript Frameworks/Libraries (e.g., React, Angular, Vue.js):** Built on top of JavaScript, these provide pre-written code and structures to streamline the development of complex, interactive user interfaces and Single Page Applications (SPAs).
-

4. Server-Side Components

Server-side refers to the **background processes** that run on the web server. This includes processing user requests, interacting with databases, and executing business logic.

- **Python:** A versatile, high-level programming language known for its clean syntax and extensive libraries. Widely used in web development (with frameworks like Django and Flask), data analysis, and artificial intelligence.
 - **PHP (Hypertext Preprocessor):** A server-side scripting language specifically designed for web development. It is one of the most widely used languages for creating dynamic web pages and is the foundation of popular content management systems like WordPress.
 - **ASP.NET:** A framework developed by Microsoft for building web applications and services. It allows developers to use languages like C# to create robust and scalable web applications.
-

5. Database Components

Databases are essential for **storing, managing, and retrieving data** used by a web application (e.g., user information, product catalogs, or content).

- **Relational Databases (SQL):** Store data in a structured format using tables with rows and columns. They use SQL (Structured Query Language) for data manipulation.
 - **Examples:** MySQL, PostgreSQL, Microsoft SQL Server.
- **Non-Relational Databases (NoSQL):** Store data in a non-tabular format such as key-value pairs, documents, or graphs. They are more flexible and scalable, making them suitable for large-scale and unstructured data applications.
 - **Examples:** MongoDB, Redis, Cassandra.

Weeks 2 & 3: Advanced Topics

1. 3-Tier Architecture

3-tier architecture is a well-established **software architecture pattern** that organizes an application into **three logical and physical tiers**. This separation of concerns makes applications more scalable, maintainable, flexible, and secure.

The three tiers are:

1. **Presentation Tier**
2. **Application Tier**
3. **Data Tier**

Key Principle: The presentation tier and the data tier cannot communicate directly; all communication must pass through the application tier. This modularity allows different teams to work on each tier simultaneously and for any tier to be updated or scaled independently without impacting the others.

- **Examples:** ibm.com

a) Presentation Tier (Client)

This is the **user interface (UI)** and the topmost layer of the application that the end-user interacts with. Its primary role is to display information to the user and collect their input. This tier can be a **web browser, a desktop application, or a mobile app**.

- **Examples:** medium.com

How it works:

The presentation tier communicates with the application tier, often through **API calls**, to send user requests and display the results. For example, when you fill out a login form and click "**submit**," the presentation tier sends this information to the application tier for processing.

- **Examples:** techtarget.com

Technologies Used:

- **HTML, CSS, and JavaScript:** Core web technologies. HTML structures the content, CSS styles it, and JavaScript adds interactivity.
 - **Examples:** ardentisys.com
 - **Frontend Frameworks/Libraries:** React, Angular, Vue.js — for building complex and dynamic user interfaces more efficiently.
 - **Examples:** vfunction.com
 - **Mobile/Desktop Technologies:** Native mobile/desktop development using languages like **Swift (iOS)** or **Kotlin (Android)**.
 - **Examples:** vfunction.com, ardentisys.com
-

b) Application Tier (Logic / Middle Tier)

This tier is the “**brain**” of the application. It contains the **business logic** (rules and processes that handle user requests). It processes data from the presentation tier, makes decisions, and communicates with the data tier to store or retrieve information.

- **Examples:** vfunction.com

How it works:

Continuing the login example — the application tier receives the **username and password** from the presentation tier. It then executes the logic to verify these credentials, often by querying the data tier. After processing, it sends a response back to the presentation tier (e.g., “**login successful**” or “**invalid credentials**”).

- **Examples:** geeksforgeeks.org

Technologies Used:

- **Programming Languages:** Python, Java, PHP, Ruby, C# (.NET).
 - **Examples:** ibm.com

- **Web Frameworks:** Django & Flask (Python), Spring (Java), Laravel (PHP), ASP.NET (C#).
 - **Examples:** ibm.com
 - **Application Servers:** Host and run the application code. Examples include **Apache Tomcat, Nginx, Microsoft IIS**.
 - **Examples:** geeksforgeeks.org, ibm.com
-

c) Data Tier (Server / Database Tier)

This tier is responsible for **storing and managing the application's data**. It consists of one or more **databases** and the **Database Management System (DBMS)** that controls access to the data. The data tier ensures **data integrity** and provides a secure way to store, retrieve, and update information.

- **Examples:** ibm.com

How it works:

The data tier only responds to requests from the **application tier**. In the login example, upon receiving a request, the database searches for the user's record and returns it. By isolating the database, **security is enhanced**, as users cannot access it directly.

- **Examples:** ibm.com

Technologies Used:

- **Relational Databases (SQL):** Used for structured data.
 - **Examples:** MySQL, PostgreSQL, Oracle, Microsoft SQL Server, techtarget.com
- **NoSQL Databases:** Used for unstructured or semi-structured data, offering high scalability.
 - **Examples:** MongoDB, Cassandra, Redis, techtarget.com

2. The 7 ISO/OSI Layers

The **Open Systems Interconnection (OSI) model** is a conceptual framework that standardizes the functions of a telecommunication or computing system into **seven abstract layers**. This model helps in understanding how data is transferred from an application on one computer to an application on another.

- **Examples:** amazon.com

Here is a description of each layer, starting from the highest level (**Layer 7**) down to the lowest (**Layer 1**).

Layer 7: Application Layer

- **Function:** The top-most layer, closest to the end-user. It provides **network services directly to applications**, acting as the interface between software and the network. Handles tasks like **file transfers, email, and web browsing**.

Note: Software applications (e.g., browsers) are not part of this layer but use its protocols.

- **Protocol Example: HTTP (Hypertext Transfer Protocol)**
 - **Relation to Programming:**
Developers use HTTP for **web applications**. For example, a Python developer might use the `requests` library to send HTTP GET/POST requests to an API.
 - **Examples:** codecademy.com, youtube.com
-

Layer 6: Presentation Layer

- **Function:** Acts as the **translator of data** for the network. Responsibilities include:

- Data formatting
 - Character encoding (e.g., ASCII → Unicode)
 - Compression
 - Encryption/Decryption
 - **Protocol Example: SSL/TLS (Secure Sockets Layer / Transport Layer Security)**
 - **Relation to Programming:**
Programmers use libraries like **OpenSSL** to secure data in transit. Installing TLS certificates on a web server enables **HTTPS**, encrypting data between client and server.
 - **Examples:** wikipedia.org, studytonight.com
-

Layer 5: Session Layer

- **Function:** Establishes, manages, and terminates **communication sessions** between devices. It handles **dialogue control, session checkpointing, and recovery** (resuming interrupted sessions).
 - **Protocol Example: RPC (Remote Procedure Call)**
 - **Relation to Programming:**
RPC frameworks (e.g., **gRPC, Thrift**) let a program on one computer run a procedure on another as if it were local. The framework manages the underlying session handling.
 - **Examples:** imperva.com
-

Layer 4: Transport Layer

- **Function:** Provides **end-to-end communication**. Responsible for **segmentation** of data and ensuring correct reassembly at the receiving end. Supports:
 - **TCP:** Reliable, connection-oriented
 - **UDP:** Faster, connectionless

- **Protocol Example: TCP (Transmission Control Protocol)**
 - **Relation to Programming:**
Developers choose between **TCP (reliable)** and **UDP (fast)** depending on application needs. Example: A file transfer app uses TCP sockets; online gaming often uses UDP.
 - **Examples:** imperva.com, oreilly.com
-

Layer 3: Network Layer

- **Function:** Handles **logical addressing, routing, and forwarding** of data packets between different networks. Uses **IP addresses** to identify devices.
 - **Protocol Example: IP (Internet Protocol)**
 - **Relation to Programming:**
Developers typically interact indirectly with IP. When creating a socket, they specify an **IP address and port**; the OS networking stack builds and routes the packets.
 - **Examples:** imperva.com, geeksforgeeks.org
-

Layer 2: Data Link Layer

- **Function:** Provides **reliable node-to-node transfer** over a physical link. Responsible for:
 - **Framing** (dividing data into frames)
 - **Physical addressing** using **MAC addresses**
 - **Error detection/correction** within a LAN
- **Protocol Example: Ethernet**
- **Relation to Programming:**
Programmers rarely interact directly with this layer; handled by NIC drivers and firmware. However, it enables LAN devices to communicate using MAC addresses.

- **Examples:** techtarget.com, geeksforgeeks.org
-

Layer 1: Physical Layer

- **Function:** Lowest OSI layer. Handles **raw transmission of bits (0s and 1s)** over physical media (cables, fiber optics, wireless). Defines **electrical, mechanical, and procedural specs** for connections.
- **Protocol Example: Ethernet (Physical aspects)**
- **Relation to Programming:**
This layer is **hardware-based** and furthest from application programming. Though not directly controlled by developers, application performance depends on the **speed and reliability** of physical connections.

3. Types of Servers

A server is a computer or system that provides resources, data, services, or programs to other computers, known as clients, over a network. In the context of web hosting, a **web server** specifically refers to software that serves web content (like web pages, images, and videos) to users.

Common types of web server software:

- **Apache HTTP Server**
Developed by the Apache Software Foundation, Apache is one of the oldest and most widely used open-source web servers. It is known for its flexibility, powerful features, and modular structure, which allows for easy customization. It runs on almost any operating system, including Linux, Windows, and macOS. (*milesweb.in*)
- **Nginx**
Pronounced "*Engine-X*", Nginx is a high-performance open-source web server known for its speed, stability, and low resource consumption. It uses an event-driven architecture, making it excellent at handling a large number of concurrent connections, ideal for high-traffic websites. (*medium.com*)
- **Microsoft Internet Information Services (IIS)**
IIS is a web server developed by Microsoft for use with the Windows NT family. It is tightly integrated with the Windows operating system and is a popular choice for hosting ASP.NET web applications and other Microsoft technologies. (*milesweb.in*)

- **LiteSpeed Web Server**

A high-performance, commercial web server often used as a drop-in replacement for Apache. It is known for being faster and more efficient than Apache, especially for PHP-based websites like WordPress. (*milesweb.in*)

- **Node.js**

While technically a JavaScript runtime environment, Node.js can be used to build web servers and network applications. Its event-driven, non-blocking I/O model makes it highly efficient for real-time applications like chat apps and online games. (*cantech.in*)

4. Companies – Web Hosting Companies

Web hosting companies provide the technologies and services needed for a website to be viewed on the internet. They rent out space on a server or a cluster of servers where you can store your website's files.

Popular web hosting companies:

- **GoDaddy**

One of the world's largest domain registrars and web hosting companies. Services include shared, VPS, and dedicated hosting, as well as a website builder and e-commerce solutions. (*hostingseekers.com*)

- **Bluehost**

Founded in 2003, Bluehost is especially popular within the WordPress community, as it is officially recommended by WordPress.org. Offers shared, WordPress, VPS, and dedicated hosting. (*wikipedia.org*)

- **Hostinger**

Known for affordable and high-performance hosting solutions. Offers shared, cloud, VPS, and managed WordPress hosting, often at competitive prices. (*medium.com*)

- **Amazon Web Services (AWS)**

A leader in cloud computing and the largest web hosting provider by market share. Offers a vast range of services, from simple website hosting to full-scale cloud infrastructure. (*hostingseekers.com*)

- **SiteGround**

Known for excellent customer support and reliable performance. Particularly popular with small businesses and WordPress users, leveraging Google Cloud infrastructure for speed and reliability. (*techradar.com*)

5. Visit a Website and Identify the Services They Offer

Example: Hostinger

Visiting Hostinger's website reveals a comprehensive suite of services for users ranging from beginners to enterprises. (*medium.com*)

Core Services:

- **Web Hosting**
 - *Shared Hosting*: Affordable option for new websites and small projects. (*quora.com*)
 - *Cloud Hosting*: Offers scalability and high performance. (*wikipedia.org*)
 - *VPS Hosting*: Provides more control and dedicated resources. (*quora.com*)
 - *Managed WordPress Hosting*: Optimized for WordPress with speed, security, and support. (*quora.com*)
- **Domain Registration**

Users can search and register domain names. Many hosting plans include a free domain for the first year. (*quora.com*)
- **Email Hosting**

Professional email hosting with custom domain addresses. (*truehost.com*)
- **Website Builder**

AI-powered, user-friendly builder for creating websites without coding knowledge. (*wikipedia.org*)
- **Security Features**

Includes free SSL certificates, DDoS protection, and regular backups. (*medium.com*)

6. Simulation: Book a Domain Name

Booking a domain is straightforward. Example using **GoDaddy** or **Hostinger**:

1. Choose a Domain Registrar

Select an ICANN-accredited registrar (e.g., GoDaddy). (*godaddy.com*)

2. Search for Your Desired Domain

Use the registrar's search bar (e.g., *my-new-awesome-website.com*). (*godaddy.com*)

3. Review Availability and Alternatives

- If available → Add to cart. (*forbes.com*)
- If taken → Alternatives suggested (e.g., *.net*, *.org*, *.ai*). (*godaddy.com*)

4. Register the Domain

- Choose registration period (1–10 years). (*godaddy.com*)
- Optional add-ons: privacy protection, email hosting.
- Provide contact + payment info to complete. (*forbes.com*)

5. Confirmation and Management

After purchase, the domain is registered in your name. You can manage it via your dashboard (DNS, email, hosting). (*forbes.com*)

9. OO Modelling of Web Applications Using UML

Object-Oriented (OO) Modelling is a method of designing software systems by viewing them as real-world objects.

UML (Unified Modeling Language) provides a standardized set of diagrams to visualize, specify, and document the design of an OO system.

For web applications, UML is useful for planning both the **structure** and **behavior** before coding begins.

Key UML Diagrams for Web Applications

1. Use Case Diagram

- **Purpose:** Shows how different users (*actors*) interact with the system, helping to define functional requirements.

- **Example:**
 - *Actors:* Customer, Administrator
 - *Customer Use Cases:* Search for Product, Add to Cart, Checkout

2. Class Diagram

- **Purpose:** Depicts the static structure of the system—classes, their attributes (data), methods (functions), and relationships.
- **Example:**
 - *Classes:* Customer, Product, Order, ShoppingCart
 - *Customer:* Attributes → name, email; Methods → login(), logout()
 - *ShoppingCart:* Method → addProduct(Product)

3. Sequence Diagram

- **Purpose:** Illustrates how objects interact over time to complete a specific task, useful for visualizing flow of logic.
- **Example:**
 - For a *Checkout* use case:
 - Customer object → calls `checkout()` on ShoppingCart
 - ShoppingCart → creates Order object
 - ShoppingCart → interacts with PaymentGateway to process payment